

Diseño Arquitectónico: Analizador Léxico

Documento de Especificación de Software

Asignatura: Compiladores

Grupo: 5

Equipo: 02

Lenguaje: Python

El presente documento detalla la arquitectura de software, las decisiones de diseño y los modelos de datos para la construcción de un analizador léxico (Scanner) escrito desde cero. El desarrollo se rige bajo la restricción estricta de no utilizar generadores léxicos automatizados (como FLEX o PLY), exigiendo la implementación manual de un Autómata Finito Determinista (DFA).

1. Estructura del Paquete y Módulos

Para cumplir con los lineamientos de empaquetado y buenas prácticas de Ingeniería de Software exigidas, la aplicación se encapsula en el espacio de nombres `unam.fi.compilers.g5.02`. Se ha optado por una arquitectura altamente modular para separar la entrada/salida, el motor léxico y las estructuras de datos.

```
unam/fi/compilers/g5/02/  
├── __init__.py  
├── main.py # Orquestador principal  
├── io/  
│   ├── reader.py # Carga completa en memoria (read())  
│   └── writer.py # Emisión de reportes (Consola + Archivo txt)  
├── lexer/  
│   ├── scanner.py # Motor de iteración y Panic-Mode Recovery  
│   └── dfa.py # Lógica de transiciones (Grafo de funciones)  
├── symbol_table/  
│   └── symtab.py # Gestión de diccionario estructurado  
└── models/  
    └── token.py # Definición de tupla inmutable de clasificación
```

Argumentación de la Arquitectura:

La separación en submódulos permite el principio de responsabilidad única. Se decidió que el submódulo `io/` realice la carga completa del archivo en memoria mediante `read()` en lugar de usar un sistema de doble búfer. Esto se justifica porque los archivos de prueba requeridos son extremadamente pequeños y la lectura directa en Python optimiza drásticamente la velocidad de ejecución y simplifica el manejo de los punteros de lectura *look-ahead*.

Argumentación de la Salida:

El módulo `writer.py` se diseñó para generar una salida dual: imprime en `stdout` exactamente el formato minimalista requerido por la evaluación, y simultáneamente genera un archivo de texto. Esto facilita documentar la evidencia en el repositorio de GitHub y adjuntarla directamente al reporte técnico en Overleaf.

2. Definición de Tokens y Autómata (DFA)

Se ha seleccionado un **Enfoque Minimalista Extensible**. Aunque el equipo posee la gramática formal completa de Solidity, el autómata actual implementa exclusivamente las expresiones regulares necesarias para satisfacer los ejemplos obligatorios de evaluación, agrupándolos estrictamente en las siguientes siete categorías.

Categoría	Expresión Regular Interna	Ejemplos Reconocidos
Keyword	<code>^(printf int)\$</code>	<code>printf</code> , <code>int</code>
Identifier	<code>^[a-zA-Z_][a-zA-Z0-9_]*\$</code>	<code>main</code> , <code>a</code>
Operator	<code>^(= \+ - * /)\$</code>	<code>=</code> , <code>+</code> , <code>*</code>
Constants	<code>^[0-9]+\$</code>	<code>10</code> , <code>256</code>
Literals	<code>^".*"\$</code>	<code>"This is an example"</code>
Punctuators	<code>^(\\(\\) ; ,)\$</code>	<code>(</code> , <code>)</code> , <code>;</code> , <code>,</code>
Special_Characters	<code>^(\\\$ @ #)\$</code>	<code>\$</code> , <code>@</code> (Capturados para Panic-Mode)

Argumentación del DFA (Grafo de Funciones):

Para la construcción manual del escáner, el módulo `dfa.py` se diseñó utilizando el *State Pattern* en lugar de una matriz de transiciones bidimensional. Cada estado es una función en Python (ej. `estado_leyendo_id()`). Esto se eligió porque evita el acoplamiento rígido de una matriz estática y facilita inyectar reglas complejas de Solidity en el futuro sin reescribir la tabla.

3. Estrategia de Manejo de Errores Léxicos

El escáner implementa la técnica de **Panic-Mode Recovery** para el manejo de excepciones a nivel léxico. Cuando el DFA encuentra un carácter que no coincide con ninguna transición válida hacia un estado de aceptación, el flujo de análisis no se detiene permanentemente.

El motor aísla el carácter ilegal, emite una advertencia de error por consola especificando la línea y columna, descarta dicho lexema malformado, y reanuda exhaustivamente la búsqueda a partir del siguiente carácter. Esto garantiza encontrar todos los errores en una sola pasada de compilación.

Argumentación sobre identificadores estandarizados:

Al confirmarse que el caso de prueba es `int a = 10;` (sin el carácter especial `$`), el DFA se simplifica para aceptar únicamente identificadores estándar (C/Solidity-like), que inician con letras o guion bajo. Cualquier carácter extraño como `$` , `@` o `#` caerá directamente en el flujo de *Panic-Mode*, garantizando un escaneo determinista y robusto.

4. Tabla de Símbolos e Inserción Temprana

Se diseñó una política de **Inserción Temprana (Fase Léxica)** mediante las operaciones `Insert()` y `Lookup()`. En lugar de delegar el trabajo al analizador sintáctico, tan pronto el lexer reconoce una cadena alfanumérica, realiza una búsqueda en el diccionario del módulo `syntab.py`. Si el lexema existe en el sub-diccionario de palabras reservadas, se emite un `Keyword`; si no, se inserta como `Identifier`.

Para la estructura de la tabla, se eligió un **Modelo de Diccionario de Objetos Estructurados**. Aunque el lexer solo necesita saber que es un identificador, se reservan los espacios exactos en memoria para los requerimientos de las fases semánticas y de generación de código.

Name	Type	Size	Dimension	Line of declamation	Line of usage	Address
count	int	2	0	TBD	TBD	0xFF...
a	TBD	TBD	TBD	2	TBD	TBD

Argumentación Estructural y Extensibilidad:

Al asignar valores "TBD" (To Be Determined) en la fase léxica, la tabla ya queda preparada para cuando el Parser y el Analizador Semántico requieran validar inferencias lógicas de predicados o generar el "Three Address Code". Además, este diseño permite escalar el lexer a Solidity en el futuro con cero esfuerzo: bastará con agregar las 80+ palabras clave de Solidity al diccionario inicial de `syntab.py`.

5. Propuesta de CFG y Left Factoring

Como requerimiento teórico, se presenta el modelo de la Gramática Libre de Contexto (CFG) completo con sus reglas de transición para las instrucciones `printf("...");` e `int a = 10;`, aplicando factorización por la izquierda para prepararla para un parser predictivo LL(1).

CFG Completo Original (No Determinista):

```
S → Statement
Statement → Keyword ( ArgList ) ; | Keyword Identifier Operator Expression ;
ArgList → Literals | Literals , Identifier
Expression → Constants | Identifier
```

Factorización por la izquierda (Left Factoring) y Transiciones: Se elimina la ambigüedad en `ArgList` extrayendo el prefijo común α = `Literals` y en `Statement` si se compartiera un prefijo.

```
S → Statement
Statement → Keyword StatementTail
StatementTail → ( ArgList ) ; | Identifier Operator Expression ;
ArgList → Literals ArgTail
ArgTail → , Identifier | ε
Expression → Constants | Identifier
```

Esta transformación garantiza que un parser predictivo pueda seleccionar la producción correcta basándose únicamente en el conjunto de símbolos *FIRST()* y el elemento *look-ahead*, eliminando cualquier necesidad de *backtracking*.

6. Instrucciones para el Desarrollador (Guía de Implementación)

Esta sección define las directrices técnicas obligatorias para el programador asignado a codificar el analizador léxico, asegurando que la implementación respete la arquitectura diseñada y las restricciones de evaluación teóricas.

A. Qué USAR (Herramientas Permitidas)

- **Patrón de Estado (State Pattern):** Implementar el autómata en `dfa.py` utilizando métodos o funciones puras que representen los estados (ej. `estado_inicial(char)`, `estado_entero(char)`).
- **Métodos de Cadenas Nativos:** Usar funciones nativas de Python como `char.isalpha()`, `char.isdigit()` y `char.isspace()` para evaluar las transiciones carácter por carácter mediante bloques `if/elif`.
- **Lectura en Bloque:** Usar `open(path).read()` en el módulo `io/reader.py` y mantener un puntero (índice numérico) que avance sobre la cadena.
- **Data Classes o NamedTuples:** Para definir la estructura inmutable del objeto `Token` en `models/token.py`.

Nota Crítica sobre Expresiones Regulares:

Las expresiones regulares (Regex) documentadas en la sección 2 son representaciones *matemáticas* de la gramática. **ESTÁ PROHIBIDO** usar el módulo `import re` de Python para buscar tokens de forma automática (ej. `re.findall()`). El objetivo de un escáner manual (Hand-written Scanner) es iterar sobre la cadena, procesando explícitamente el cambio de estado con cada nuevo carácter leído.

B. Qué NO USAR (Prácticas y Herramientas Prohibidas)

- **Generadores Léxicos Automáticos:** Estrictamente prohibido instalar o utilizar **FLEX**, **Lex**, **PLY**, **SLY** u otras librerías que automaticen la generación de la máquina de estados.
- **Variables Globales Flotantes:** Evitar el uso de variables globales sueltas. El estado general del escáner (número de línea actual, columna actual, puntero de lectura) debe estar encapsulado dentro de una clase principal `Scanner`.
- **Bucle infinito sin mecanismo de salida:** Asegurarse de procesar e identificar correctamente el carácter especial de fin de archivo `EOF` (End Of File) para terminar el ciclo de lectura.

C. Ruta de Desarrollo y Lógica de la Máquina de Estados (DFA)

Para construir un **Autómata Finito Determinista (DFA) funcional** que lea el código fuente y extraiga los tokens correctamente, el desarrollador debe seguir esta estructura lógica iterativa:

1. **Bucle de Escaneo Principal:** En `scanner.py`, inicializa un bucle `while pointer < len(source_code)`. El analizador debe moverse estrictamente carácter por carácter sobre el búfer de entrada.
2. **Transición de Estados (State Pattern):**
 - Inicia siempre cada iteración principal evaluando el carácter actual en el `estado_inicial()`.
 - **Reglas de Transición y Acumulación:**
 - Si `char.isalpha()` o `char == '_'`: Entrar al `estado_leyendo_identificador()`. Sigue consumiendo caracteres mientras sean alfanuméricos o guiones bajos. Al encontrar un delimitador (espacio o puntuador), detente, extrae el lexema y aplica `Lookup()` para emitir **Keyword** o **Identifier**.
 - Si `char.isdigit()`: Entrar al `estado_leyendo_constante()`. Consume números hasta encontrar un carácter no numérico. Emite el token **Constants**.
 - Si `char == '"'`: Entrar al `estado_leyendo_literal()`. Consume todos los caracteres ignorando reglas anteriores hasta encontrar la comilla de cierre `"`. Emite **Literals**.
 - Si `char` es operador (ej. `=`, `+`): Emite directamente **Operator**.
 - Si `char` es puntuación (ej. `;`, `(`): Emite directamente **Punctuators**.
 - Si `char` es un símbolo ajeno (ej. `$`): Clasifícalo temporalmente como **Special_Characters** y lanza el protocolo de manejo de errores (*Panic-Mode Recovery*) registrando la línea y descartándolo de los lexemas válidos.
3. **Control de Punteros (Look-ahead):** Ten extremo cuidado de no "comerse" un carácter válido al salir de un estado complejo. Por ejemplo, al leer `int a=10;`, el estado de constante lee `1` y `0`, el siguiente carácter es `;`. El bucle de `estado_leyendo_constante()` debe detenerse al ver el `;` sin consumir ese carácter, dejándolo intacto para que el `estado_inicial()` lo atrape en la siguiente iteración como **Punctuators**.